

Explanation

Lab-3

Task 1

[9, 5, 4, 6, 1, 3, 2, 9]

mid

This task is about merge sort. Here at first we take a mid index and divide the array left and right part. I am taking three pointers i, j, k . Starting from first index at both left and right we will compare which is smaller. If it's smaller than other the we will put it in our new array. If it's done recursively the whole array will be sorted. Now if one array is over and other is going what will happen? The answer is if left array ends we simply put the right array numbers serially and if the right array ends it's vice-versa.

Task-2:

Approach is similar to the task 1 but ~~here~~ here I can't sort the array. So here I will take a $\text{max_val} = 0$. I'll compare with my recursive call at left and right. If it gives bigger value the maximum will be updated. At the end we will find our ~~max~~ max without sorting the array.

The time complexity of the code is $O(n \log n)$.

Task-4:

Here at first I made array of squared values. So that I can pick its maximum squared. As, $i < j$ so from the squared array I can't take the first value. Used a max index

function to get our j value and index. As $i > j$, so after getting j I sliced array from i to j th value, ~~so then~~ then sorted the array and took the biggest value. Then we get our i and j . Then ~~so~~ simply added both values.

Task-5

	^{pivot} ↑								
	[	9	5	4	6	1	3	2	9]
		low			mid				high

To do quick sort first we will ~~recursion~~ call the partition function. We take a pivot = $arr[p]$, and pointer i and j . If we get the value less than pivot we increase i . If its less than j we decrease j . If j pointer passes i pointer we swap i and j . After loop breaks we swap pivot with that index. We will return that index.

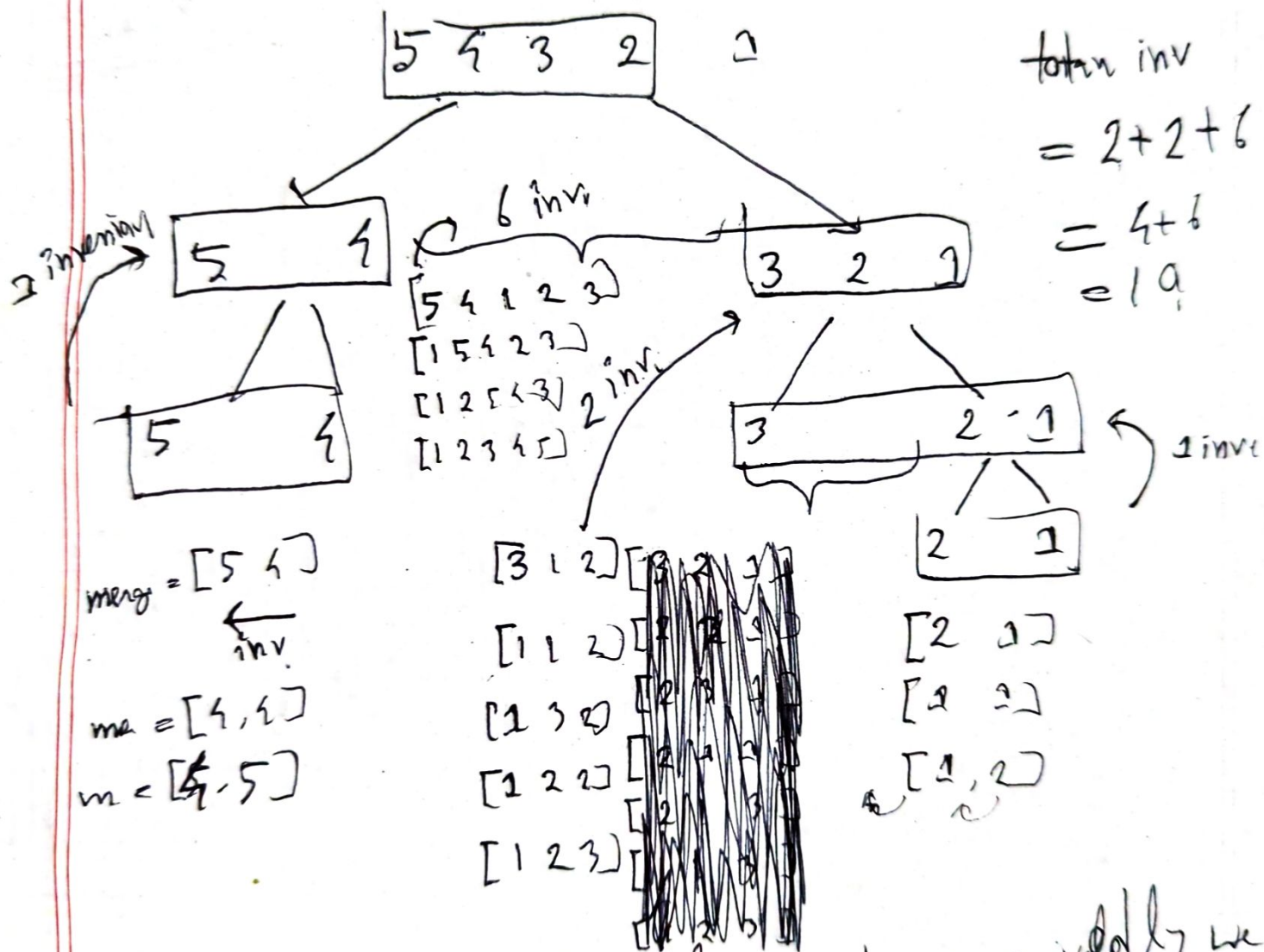
and recursively call partition again until its sorted.

Task-6

This task is similar to task 5. Here we just don't sort the array. To get out k^{th} index we used partition function where we will return index that swaps with our pivot if it matches with ~~not~~ our k value then it's the answer. If it is smaller than k then we will check ^{from} the $p+1$ index and if it's bigger we check from low to p index.

Task-3

Task was using divide and conquer method. Here I took a variable `inv` which will count the inversions.



merged all left and right and recursively we got our inversion result.